National University of Computer and Emerging Sciences

Lahore Campus

Data Structures Lab

BCS-3G/5G

Date: 15th October, 2026

Instructor

Ahmad Jawad Mustasim

Sessional Exam

Total Time (Hrs): 2

Total Marks: 50

Total Questions: 2

Instructions:

- Create a separate file for each task, and zip the .cpp file of each task along with its output screenshot. Name the zip file with your roll number (e.g., 23L123.zip).
- Submit .zip file inside //exam /Fall 2026 Mid Exam /Data Structures /BCS-3G /Submissions

---

Attempt all the questions.

Helper code for both questions is provided. Copy it from the folder.

Question 1 – [38 marks] – LLO2

A teacher maintains a linked list of student acknowledgments for various academic occasions. Some students appear multiple times, while others have no acknowledgment record at all.

The system must identify missing records, eliminate duplicates, rearrange existing records, and verify the final registry.

One important rule applies throughout the system:

A record that was never submitted cannot be recovered through sorting, searching, or recursion.

Implement all required operations recursively. The program must manipulate linked-list nodes directly rather than relying on arrays or STL containers.

Node structure

```
class Node {
   int studentID;
   int occasionCode;
   Node* next;
};
```

Tasks:

1 – (8 marks)

Recursive duplicate removal

Implement removeDuplicates(Node* head) recursively. Keep the first occurrence of every student ID and delete all subsequent nodes containing that ID.

Constraints:

- No loops or additional containers.
- Do not swap node data.
- Free deleted nodes correctly.
- Preserve the relative order of surviving nodes.

Example: 12 → 15 → 12 → 18 → 15 → 20

Output: 12 → 15 → 18 → 20

2 – (10 marks)

Recursive missing-student detection

Using the Node structure defined above, you are given the heads of two existing linked lists. The first list contains the records of all enrolled students, while the second contains acknowledgment records submitted by students for various academic occasions. Both lists use the same Node structure.

In the enrolled-student list, the occasionCode field is set to 0 because these nodes represent student records only. In the acknowledgment list, occasionCode identifies the occasion for which each acknowledgment was submitted.

Apparently, some students remembered to attend class but forgot to wish their teacher on Teachers' Day! Help the teacher identify all enrolled students who did not submit a Teachers' Day acknowledgment.

Implement the following recursive functions:

- containsID() — recursively search the acknowledgment list for a record matching the given student ID and the Teachers' Day occasion code.
- findMissing() — recursively traverse the enrolled-student list and report every student ID for which no matching Teachers' Day acknowledgment exists.

Constraints:

- Neither original linked list may be modified.
- Duplicate IDs in the acknowledgment list must not affect the result.
- All searches and traversals must be performed recursively.

Remember: A missing acknowledgment cannot be recovered through recursion, no matter how hard the teacher wishes for it!

3 – (8 marks)

Recursive stable partitioning

Move all nodes with even student IDs to the end of the list while preserving the original relative order of even and odd nodes.

Example:

11 → 14 → 13 → 16 → 17 → 20 → 19

Output:

11 → 13 → 17 → 19 → 14 → 16 → 20

Constraints:

- Recursion only.
- No new data nodes.
- No swapping student IDs.
- Preserve valid head and tail pointers.
- Handle empty, all-even, and all-odd lists.

4 – (5 marks)

Recursive reversal

Reverse the linked list using recursion only.

Function prototype:

```
Node* reverseList(Node* head);
```

No loops, auxiliary linked lists, or swapping of data are allowed. The function must return the new head and correctly handle an empty list and a single-node list.

5 – (7 marks)

Recursive registry comparison

Implement:

bool identical(Node* a, Node* b);

Return true only if both lists contain exactly the same sequence of student IDs and occasion codes.

Lists of different lengths must return false, even when their common prefix is identical. Do not modify either list.

Mandatory restrictions

- No for, while, or do-while loops in any student-defined operation.
- No arrays, vectors, sets, maps, or other STL containers for storing or processing the records.
- No swapping node data to simulate pointer manipulation.
- All traversals must be recursive.
- Deleted nodes must be released safely.

Question 2 – [12 marks] – LLO2

Smart Undo/Redo System Using Linked-List Stacks

A text editor must support undo and redo operations without storing a complete copy of the document after every change.

Implement a basic undo/redo system using linked-list stacks to maintain the history of editing operations.

Requirements

1. **Insert and Delete Operations (4 Marks)**
    - Implement insertion of a character at a specified position.
    - Implement deletion of a character at a specified position.
    - Reject invalid positions without modifying the document.
2. **Undo and Redo Operations (4 Marks)**
    - Implement undo() to reverse the most recent editing operation.
    - Implement redo() to reapply the most recently undone operation.
    - Store only the information required to reverse or reapply an operation.
    - Correctly restore the document to its previous state when undoing an insertion or deletion.

3. **History Management (2 Marks)**
    - Use linked-list stacks to maintain undo and redo histories.
    - When a new editing operation is performed after an undo, clear the redo history.
4. **Boundary Conditions (2 Marks)**
    - Handle empty undo and redo stacks safely.
    - Ensure that invalid operations do not corrupt the document or history.

Mandatory Restrictions

- Implement undo and redo histories using linked-list stacks.
- Do not store complete document copies for every editing operation.
- Release dynamically allocated memory appropriately.
- Demonstrate the system using a sequence of insert, delete, undo, and redo operations.

Marks distribution — 50 marks

Question 1: Recursive Linked-List Operations (38 marks)

Task	Topic	Marks
1	Recursive duplicate removal	8
2	Recursive missing-student detection (containsID, findMissing)	10
3	Recursive stable partitioning	8
4	Recursive reversal (reverseList)	5
5	Recursive registry comparison (identical)	7
	Total	38

Question 2: Smart Undo/Redo System (12 marks)

Task	Topic	Marks
1	Insert and delete operations	4
2	Undo and redo operations	4
3	Redo history invalidation	2
4	Boundary conditions and safe handling	2
	Total	12

----------Good luck! ☺----------